Objective Overview: Python + Algorithm Fundamentals for Amazon SDE Assessment

Getting Started with Python
Python is a high-level, dynamically-typed programming language known for its readability and simplicity. It's ideal for interviews because it allows you to write less code and focus on solving problems efficiently. You don't need to compile code manually, and its built-in functions and data structures make it fast to prototype solutions.

Python Syntax
Python uses indentation (spaces or tabs) to define blocks of code, unlike other languages that use braces. Statements end at the newline; semicolons are optional. Functions are defined with 'def', and variables do not require explicit type declarations.

Arrays (Lists in Python)
Arrays are ordered collections of items, called 'lists' in Python. They are mutable, can hold mixed data types, and support operations like indexing, slicing, appending, and iteration. They are ideal for storing sequences of data.

Strings
Strings are sequences of characters and are immutable in Python. You can access individual characters using indexes, slice substrings, and use powerful built-in string methods for manipulation, comparison, searching, and formatting.

Hash Maps (Dictionaries in Python)
Hash maps store key-value pairs and are implemented as 'dict' in Python. They allow for fast lookups, insertions, and deletions (average O(1) time). They're commonly used for frequency counts, caching, and quick access to data by key.

Sliding Window
The sliding window is a technique used to optimize problems involving arrays or strings. It maintains a subset of elements (a window) and slides it across the data to track sums, lengths, or other properties in O(n) time.

Binary Search
Binary search is a fast search algorithm for sorted arrays. It repeatedly divides the search interval in half, achieving O(log n) time complexity. It's ideal for finding specific values or insert positions efficiently.

Recursion / Backtracking
Recursion is a technique where a function calls itself to solve subproblems. Backtracking is a recursive approach used to explore all possible configurations, especially in problems like combinations, permutations, and constraint satisfaction.

Tree Traversal (DFS & BFS)
DFS (Depth-First Search) and BFS (Breadth-First Search) are methods for traversing tree or graph structures. DFS explores as deep as possible along branches (often implemented with recursion), while BFS explores level by level (using a queue). These are essential for understanding hierarchical data and are widely tested in interviews.

Amazon SDE Technical Topics Overview (with Python Examples)

Arrays & Strings
📘 Definition: Arrays and strings are fundamental data structures used to store sequences of elements or characters. In Python, arrays are represented as lists, and strings are immutable character sequences.

💻 Example Usage (Python):

```
# Array (list)
nums = [1, 2, 3]
nums.append(4)
print(nums[1])  # Output: 2

# String
s = "amazon"
print(s.upper())  # Output: AMAZON
print(s[::-1])    # Output: nozama
```

Hash Tables / Sets
📘 Definition: Hash tables (dictionaries) and sets provide fast lookups and are ideal for counting, indexing, or membership checking in constant average time complexity (O(1)).

💻 Example Usage (Python):

```
# Dictionary (hash map)
freq = {}
for char in "banana":
    freq[char] = freq.get(char, 0) + 1
print(freq)  # Output: {'b': 1, 'a': 3, 'n': 2}

# Set
seen = set()
seen.add("apple")
print("apple" in seen)  # Output: True
```

Recursion / Backtracking
📘 Definition: Recursion is when a function calls itself to solve a subproblem. Backtracking is a technique that incrementally builds candidates and abandons those that fail to satisfy constraints.

💻 Example Usage (Python):

```
# Backtracking: generate all subsets
def backtrack(start, path):
    result.append(path[:])
    for i in range(start, len(nums)):
        path.append(nums[i])
        backtrack(i + 1, path)
        path.pop()

nums = [1, 2]
result = []
```

```
backtrack(0, [])
print(result)  # Output: [[], [1], [1, 2], [2]]
```

Binary Trees / Graphs
📘 Definition: Binary trees are hierarchical structures where each node has at most two children. Graphs are more general networks of nodes and edges, often explored with DFS or BFS.

💻 Example Usage (Python):

```
# Binary Tree DFS
def maxDepth(root):
    if not root:
        return 0
    return 1 + max(maxDepth(root.left), maxDepth(root.right))

# Graph BFS
from collections import deque

def bfs(graph, start):
    visited = set()
    queue = deque([start])
    while queue:
        node = queue.popleft()
        if node in visited:
            continue
        visited.add(node)
        for neighbor in graph[node]:
            queue.append(neighbor)
    return visited
```

Sorting / Searching Algorithms
📘 Definition: Sorting organizes data in a defined order (e.g., ascending). Searching finds an element in a collection. Binary search is a common and efficient method for sorted data.

💻 Example Usage (Python):

```
# Sorting
arr = [3, 1, 4, 1, 5]
arr.sort()
print(arr)  # Output: [1, 1, 3, 4, 5]

# Binary Search
def binary_search(arr, target):
    left, right = 0, len(arr) - 1
    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
```

```
        right = mid - 1
    return -1
```

Greedy Algorithms

📘 Definition: Greedy algorithms make the locally optimal choice at each step in hopes of finding the global optimum. They are fast but not always correct unless the problem satisfies the greedy-choice property.

💻 Example Usage (Python):

```
# Greedy: Minimum number of coins
def minCoins(coins, amount):
    coins.sort(reverse=True)
    count = 0
    for coin in coins:
        while amount >= coin:
            amount -= coin
            count += 1
    return count

print(minCoins([1, 2, 5], 11))  # Output: 3 (5+5+1)
```

Dynamic Programming (DP)

📘 Definition: DP solves problems by breaking them into overlapping subproblems and storing results to avoid recomputation. It’s used for optimization problems like min/max path, subset sum, etc.

💻 Example Usage (Python):

```
# DP: Fibonacci sequence
def fib(n):
    dp = [0, 1]
    for i in range(2, n+1):
        dp.append(dp[i-1] + dp[i-2])
    return dp[n]

print(fib(6))  # Output: 8
```

Amazon SDE Online Assessment Preparation Pack

Section 1: Python Review Cheat Sheet

```
# Lists

nums = [1, 2, 3]

nums.append(4)

nums.pop()

nums[0]


# Dictionary (Hash Map)

d = {'a': 1}

d['a']

d.get('b', 0)


# Set

s = set([1, 2, 3])

s.add(4)

s.remove(2)


# Loops

for i in range(5): print(i)

while condition: pass


# Functions

def add(a, b): return a + b


# List Comprehension

[x for x in range(5) if x % 2 == 0]


# Lambda

square = lambda x: x*x
```

Sorting

sorted_list = sorted([3, 1, 2])

sorted_pairs = sorted(pairs, key=lambda x: x[1])

Section 2: Key Problem-Solving Patterns (with Variations + Hints)

Sliding Window – Maximum Sum Subarray of Size K
📝 Prompt: Given an array of integers nums and an integer k, return the maximum sum of a subarray with exactly k elements.

📌 Example:
Input: nums = [2, 1, 5, 1, 3, 2], k = 3
Output: 9

💡 Hints:

- - Use a sliding window of size k. Start by summing the first k elements.
- - As you slide the window, subtract the element that's leaving and add the new one.

Binary Search – First Bad Version
📝 Prompt: You are given an API isBadVersion(version). Find the first bad version between 1 and n.

📌 Example:
Input: n = 5, first bad version = 4
Output: 4

💡 Hints:

- - This is a classic binary search problem. Treat 1 to n as your search space.
- - If isBadVersion(mid) is True, search the left half. You're looking for the earliest bad one.

Backtracking – Combination Sum
📝 Prompt: Given a list of candidates (distinct integers) and a target, return all unique combinations that sum to the target.

📌 Example:
Input: candidates = [2, 3, 6, 7], target = 7
Output: [[2, 2, 3], [7]]

💡 Hints:

- - Use backtracking: explore each number by choosing or skipping it.
- - Since you can reuse elements, don't move the index forward when calling recursively.

DFS – Path Sum
📝 Prompt: Given a binary tree and a target sum, return True if the tree has a root-to-leaf path that adds up to the sum.

📌 Example:
Input: root = [5,4,8,11,null,13,4,7,2,null,null,null,1], target = 22
Output: True

💡 Hints:

- - Use recursive DFS. Subtract the current node value from the target.
- - Return True only if the target becomes zero exactly at a leaf node.

BFS – Minimum Depth of Binary Tree
📝 Prompt: Given a binary tree, find its minimum depth from root to the nearest leaf.

📌 Example:
Input: root = [3,9,20,null,null,15,7]
Output: 2

💡 Hints:

- - Use BFS (level-order traversal) to find the first leaf node.

- As soon as you hit a leaf, return the current depth.

Here's a **concise and strategic written overview** of the Amazon Leadership Principles, tailored to help you prepare for the **behavioral part** of your SDE assessment:

🧭 Overview: Amazon's Leadership Principles (for SDE Candidates)

Amazon uses its **16 Leadership Principles** as the foundation of its decision-making, hiring, and culture. In the behavioral part of your assessment, your answers will be evaluated based on how well they align with these principles — especially in terms of **ownership, problem-solving, and long-term thinking**.

Below is a summarized view of the principles, grouped by relevance to **SDE candidates**, with notes on **how to apply them in behavioral questions**.

🔑 Core Principles to Emphasize in the Assessment

1. Customer Obsession

Amazon starts with the customer and works backwards.
📌 Show how you prioritized user experience or improved product outcomes in a project.

2. Ownership

Act on behalf of the entire company, not just your team.
📌 Mention times when you took initiative beyond your assigned role or followed through on long-term impact.

3. Invent and Simplify

Look for new solutions and reduce complexity.
📌 Highlight problems where you applied a novel algorithm or refactored code for clarity and performance.

4. Are Right, A Lot

Strong judgment and good instincts.
📌 Share a time you proposed a technical direction that turned out to be correct — and how you backed it up with data or logic.

5. Learn and Be Curious

Never stop learning.
📌 Talk about new frameworks, algorithms, or coding practices you taught yourself or applied.

🛠️ Execution-Focused Principles

6. Bias for Action

Speed matters.
📌 Explain how you delivered results under tight deadlines or moved quickly in ambiguous situations.

7. Deliver Results

Focus on key inputs and outcomes.
📌 Share a story where you completed a project successfully, despite challenges.

8. Dive Deep

Stay connected to the details.
📌 Describe a time when your debugging or analysis revealed a hidden issue others overlooked.

🤝 Collaboration and Leadership Principles

9. Earn Trust

Build trust through honesty, empathy, and results.
📌 Mention examples of peer communication or resolving conflict professionally.

10. Insist on the Highest Standards

Continually raise the bar.
📌 Talk about quality code reviews, unit testing, or automation you implemented.

11. Think Big

Create bold direction.
📌 Share any experience designing scalable solutions or thinking beyond your current feature/task.

🧠 Bonus Principles (Optional, but Impressive)

12. Frugality

Accomplish more with less.
📌 Show how you worked around constraints or optimized performance/resource usage.

13. Hire and Develop the Best

If relevant, mention how you mentored others or learned from strong teammates.

14. Strive to be Earth's Best Employer

Not directly tested, but can reflect empathy and support for teammates.

15. Success and Scale Bring Broad Responsibility

Think ethically about your work's impact — especially if it affects customers at scale.

16. Have Backbone; Disagree and Commit

Speak up respectfully, even when it's hard — then fully support the decision.

How to Use These in the Behavioral Test

Choose relevant examples from your academic, internship, or personal projects.

Use the **STAR** method to structure responses:
Situation → **T**ask → **A**ction → **R**esult

Pick **2–3 principles per story** and demonstrate alignment clearly.

Amazon SDE Behavioral Assessment Overview + Mock Questions

Objective Overview of Key Leadership Principles
Amazon evaluates candidates not just on technical skills, but also on how well they embody its 16 Leadership Principles. During the Work-Style Assessment and final interviews, you'll be expected to demonstrate alignment with principles such as Customer Obsession, Ownership, Invent and Simplify, and Deliver Results.

Use the STAR method (Situation, Task, Action, Result) to structure your answers. Prepare stories that showcase your ability to take initiative, solve problems efficiently, and work collaboratively in challenging environments.

Customer Obsession
Always start with the customer and work backward. Focus on creating value for users.

Ownership
Take responsibility for outcomes, even beyond your job role.

Invent and Simplify
Innovate with simple and scalable solutions.

Are Right, A Lot
Demonstrate strong judgment and technical insight.

Learn and Be Curious
Show continuous learning and eagerness to explore new areas.

Bias for Action
Make timely decisions, especially under uncertainty.

Deliver Results
Meet goals with efficiency, even under constraints.

Dive Deep
Stay close to the details and investigate thoroughly.

Earn Trust
Communicate honestly and build credibility with others.

Insist on the Highest Standards
Strive for excellence and quality.

Think Big
Pursue bold ideas and challenge assumptions.

Frugality
Accomplish more with less through smart problem-solving.

Have Backbone; Disagree and Commit
Speak up respectfully and commit to decisions once made.

Simulated Work-Style Assessment Questions

1. Tell me about a time you took ownership of a project or task that was not assigned to you.
2. Describe a situation where you had to dive deep to find the root cause of a technical issue.
3. Tell me about a time you invented or simplified a process.
4. Give an example of how you handled a disagreement within a team.
5. Describe a situation where you had to deliver results under pressure or with limited resources.
6. Tell me about a time when you made a mistake. How did you handle it?
7. Explain how you ensure that your code meets high quality standards.
8. Describe a time you had to learn a new tool or concept quickly to complete a project.
9. Tell me about a bold idea you proposed. What was the outcome?
10. Describe a time when you had to make a quick decision without all the data.

Let's start by writing **model answers** for a few of the most critical behavioral questions — aligned with Amazon's STAR method and principles.

I'll do **5 answers** for now, each one with:

- Clear structure (**Situation** → **Task** → **Action** → **Result**)
- Natural, human tone (not robotic)
- Alignment with **key leadership principles** (noted for each)

Amazon SDE Behavioral Interview – Model Answers

Tell me about a time you took ownership of a project or task that was not assigned to you.

Leadership Principles: Ownership, Bias for Action

S: During a university group project to develop a task manager app, one team member who was responsible for backend integration dropped out unexpectedly.
T: Without a functioning backend, we couldn't test or deploy the app — and the deadline was a week away.
A: I wasn't originally assigned that part, but I stepped in, studied the Firebase documentation, and integrated authentication and database services. I also coordinated with the rest of the team to ensure minimal rework.
R: We completed the app on time and received full marks. The team even mentioned that my initiative helped us stay on track.

Describe a situation where you had to dive deep to find the root cause of a technical issue.

Leadership Principles: Dive Deep, Are Right, A Lot

S: In a personal project, my API was returning inconsistent data — sometimes missing fields or crashing.
T: I needed to figure out why the issue was happening, especially since it was hard to reproduce consistently.
A: I added detailed logging and traced the request flow. I found that the bug was triggered by empty values in edge-case responses from an external service. I updated the parser to handle these cases gracefully and wrote additional unit tests.

R: After the fix, the API became stable. I now start all projects with defensive programming and logging by default.

Tell me about a time you invented or simplified a process.

Leadership Principles: Invent and Simplify, Think Big

S: While working on a team assignment, we were manually copying test results into a spreadsheet to analyze performance — very repetitive and error-prone.
T: I wanted to simplify the process to save time and reduce mistakes.
A: I wrote a Python script that parsed the test outputs and automatically generated the summary report. I shared it with the group and showed them how to use it.
R: It saved us hours of work and we used it in future projects. That experience taught me that small automations can make a big difference.

Give an example of how you handled a disagreement within a team.

Leadership Principles: Have Backbone; Disagree and Commit, Earn Trust

S: In a hackathon, our team was split on whether to focus on backend performance or add more features.
T: I believed strongly that stability mattered more than new features given the 24-hour timeframe.
A: I explained my reasoning calmly and shared an example from a previous hackathon where instability caused issues in the demo. After hearing both sides, we agreed to prioritize performance first, and if time allowed, we'd add one new feature.
R: We delivered a smooth demo with a working feature set and ended up placing second.

Describe a situation where you had to deliver results under pressure or with limited resources.

Leadership Principles: Deliver Results, Bias for Action

S: During finals week, I had both an important exam and a pending software engineering assignment.
T: I had 48 hours to deliver a working prototype and was exhausted from studying.
A: I created a focused action plan with clear milestones and time blocks. I prioritized features by impact, dropped non-critical ones, and reused code from earlier assignments where possible. I also minimized distractions and worked offline.
R: I submitted a working prototype 2 hours before the deadline. It wasn't perfect, but it was stable and met all core requirements.

Tell me about a time when you made a mistake. How did you handle it?

Leadership Principles: Earn Trust, Learn and Be Curious

S: In one project, I accidentally pushed code to the wrong branch, which affected the main codebase.
T: I needed to take responsibility and fix the issue quickly before it impacted others.
A: I notified the team immediately, reverted the commit, and double-checked all my changes. Then I created a checklist to avoid this in future commits and added a pre-push hook to prevent mistakes.
R: My team appreciated the transparency and I never repeated that error. It was a valuable learning experience.

Explain how you ensure that your code meets high quality standards.

Leadership Principles: Insist on the Highest Standards

S: In both classwork and personal projects, I'm always focused on producing clean, maintainable code.
T: I aim to write code that others can understand and build on — not just code that works.
A: I write modular functions, meaningful variable names, and test coverage for critical logic. I also use linters and static analyzers.
R: As a result, I've received positive feedback from peers during code reviews and rarely need to revisit bugs in finished modules.

Describe a time you had to learn a new tool or concept quickly to complete a project.

Leadership Principles: Learn and Be Curious

S: For a university assignment, we had to use a graph database, something I had no experience with.
T: I needed to learn Neo4j and Cypher query language in just two days.
A: I followed official tutorials, practiced examples, and built a small demo before starting on the actual task. I even shared a short guide with my teammates to help them ramp up too.
R: I successfully integrated Neo4j into our system and our final demo included relationship queries that impressed the professor.

Tell me about a bold idea you proposed. What was the outcome?

Leadership Principles: Think Big, Invent and Simplify

S: During a team discussion about how to track bugs, I suggested we automate the error reporting pipeline.
T: It wasn't part of our scope, but I believed it would save a lot of time in the long run.
A: I prototyped a logging system that captured errors and pushed them to a shared dashboard using a webhook.
R: My team adopted the tool, and it helped cut debugging time nearly in half. It also inspired other teams to try similar tools.

Describe a time when you had to make a quick decision without all the data.

Leadership Principles: Bias for Action, Are Right, A Lot

S: While presenting a class project, a key feature failed to load due to an unknown error.
T: I had only a few seconds to decide whether to troubleshoot live or skip the feature.
A: I quickly pivoted and showed a working local backup version of the feature. I explained the bug briefly, and then proceeded with the rest of the demo.
R: The presentation went smoothly, and we still scored well. I learned the importance of preparing contingencies and adapting fast.

Amazon Work-Style Simulation questions (also known as the behavioral personality assessment), with ideal answer choices and reasoning behind each. These are not long-form STAR questions — they're **forced-choice or agreement scale** questions meant to evaluate how well your natural work style fits Amazon's culture.

Amazon is looking for people who show **ownership, curiosity, action-orientation, customer focus, and high standards** — but balanced with collaboration, learning, and long-term thinking.

✅ **Format: Possible Questions + Ideal Answers + Why**

🧠 **1. I prefer making decisions quickly rather than waiting until I have all the information.**

✅ **Strongly Agree**
💡 **Why?** This reflects **Bias for Action**, one of Amazon's core principles. They want people who can move fast in ambiguity and adapt later if needed.

🎯 **2. I take responsibility for failures, even if I was only partially involved.**

✅ **Strongly Agree**
💡 **Why?** This aligns with **Ownership** — Amazon values individuals who don't shift blame and who act like "it's their company."

🚀 **3. I look for ways to raise the bar in the quality of my work.**

✅ **Strongly Agree**
💡 **Why?** This supports **Insist on the Highest Standards**. Amazon wants builders who continuously improve their work and don't settle for "good enough."

👥 **4. It's important to challenge a teammate's decision when I believe there's a better way.**

✅ **Agree**
💡 **Why?** Amazon respects **Have Backbone; Disagree and Commit** — speak up respectfully when needed, but still support the final group decision.

📊 **5. I feel comfortable making trade-offs to deliver results on time.**

✅ **Agree**
💡 **Why?** Amazon expects pragmatism — balancing perfection with deadlines. This shows **Deliver Results** and **Bias for Action** while still being realistic.

🔍 **6. I focus on the details, even when working under time pressure.**

✅ **Agree**
💡 **Why?** This shows **Dive Deep** — being detail-oriented, even when moving fast.

🧩 **7. I prefer to take time to analyze a situation fully before taking action.**

✅ **Disagree** or **Neutral**
💡 **Why?** While thoughtful analysis is good, **Bias for Action** means you're comfortable deciding

without perfect data.

🤝 **8. I would rather work alone than have to slow down for team discussions.**

✅ **Disagree**
💡 **Why?** Amazon values collaboration and **Earn Trust**. Even fast workers must contribute to strong team dynamics.

📚 **9. When I face a challenge in an area I don't understand, I learn what I need to solve it.**

✅ **Strongly Agree**
💡 **Why?** This supports **Learn and Be Curious** — core for SDEs expected to grow quickly and independently.

📦 **10. If I notice a way to improve a process, I act on it without being asked.**

✅ **Strongly Agree**
💡 **Why?** That's **Invent and Simplify** + **Ownership** in action.

💬 **Final Tip: Keep the following in mind**

When in doubt, favor:

- Taking initiative
- Being detail-oriented but action-driven
- Owning results
- Working collaboratively
- Learning independently

Avoid:

- Indecisiveness
- Blaming others
- Working in isolation
- Avoiding conflict when you have valid concerns

Amazon's Work-Style Assessment, written inline here for you to reflect and answer. After each question, I'll list the **best answer**, followed by an **explanation** — so you can test yourself first and then check your alignment with Amazon's culture.

🧪 Amazon Work-Style Practice Quiz (with Answers & Explanations)

Choose your answer:

- Strongly Agree
- Agree
- Neutral
- Disagree
- Strongly Disagree

---

1. I prefer making decisions quickly rather than waiting until I have all the information.

✅ **Best Answer:** Strongly Agree
💬 **Explanation:** Amazon values **Bias for Action** — you should be able to make fast, informed decisions even with some ambiguity.

---

2. I take responsibility for failures, even if I was only partially involved.

✅ **Best Answer:** Strongly Agree
💬 **Explanation:** This shows **Ownership** — taking accountability regardless of scope.

---

3. I look for ways to raise the bar in the quality of my work.

✅ **Best Answer:** Strongly Agree
💬 **Explanation:** Reflects **Insist on the Highest Standards** — a core Amazon principle that they evaluate heavily.

---

4. It's important to challenge a teammate's decision when I believe there's a better way.

✅ **Best Answer:** Agree
💬 **Explanation:** This supports **Have Backbone; Disagree and Commit** — voice your concerns respectfully, then commit to team decisions.

---

5. I feel comfortable making trade-offs to deliver results on time.

✅ **Best Answer:** Agree
💬 **Explanation:** This shows **Deliver Results** — balancing quality and deadlines is key.

---

6. I focus on the details, even when working under time pressure.

✅ **Best Answer:** Agree
💬 **Explanation:** Shows ability to **Dive Deep** without losing sight of execution.

---

7. I prefer to take time to analyze a situation fully before taking action.

✅ **Best Answer:** Disagree
💬 **Explanation:** While thoughtful analysis is good, Amazon wants people who are decisive — this aligns better with **Bias for Action**.

---

8. I would rather work alone than have to slow down for team discussions.

✅ **Best Answer:** Disagree
💬 **Explanation:** Amazon values **Earn Trust** and **collaboration**. Independence is important, but teamwork is essential.

---

9. When I face a challenge in an area I don't understand, I learn what I need to solve it.

✅ **Best Answer:** Strongly Agree
💬 **Explanation:** A perfect match for **Learn and Be Curious** — a standout principle for Amazon SDEs.

---

10. If I notice a way to improve a process, I act on it without being asked.

✅ **Best Answer:** Strongly Agree
💬 **Explanation:** Embodies **Invent and Simplify** and **Ownership**. Amazon loves proactive problem-solvers.

Here's a **second set** of Amazon-style Work-Style Assessment questions — this time with a few **trap answers** included to help you identify what **not** to choose and why.

Try to spot the best choice yourself first, then compare with the explanation to reinforce your intuition.

🧪 Amazon Work-Style Practice – Set 2 (Includes Trap Answers)

Choose your answer:

- Strongly Agree
- Agree
- Neutral
- Disagree
- Strongly Disagree

---

1. I usually wait for someone to assign tasks before I begin working.

❌ **Trap Answer:** Agree
✅ **Best Answer:** Strongly Disagree
💬 **Why:** This contradicts **Ownership**. Amazon wants you to act independently and take initiative, not wait for instructions.

---

2. I prefer perfect results, even if it means missing a deadline.

❌ **Trap Answer:** Strongly Agree
✅ **Best Answer:** Disagree
💬 **Why:** This conflicts with **Deliver Results** and **Bias for Action**. Amazon values timely results over perfectionism when trade-offs are required.

---

3. I tend to avoid giving feedback if I think it will upset a teammate.

❌ **Trap Answer:** Agree
✅ **Best Answer:** Disagree
💬 **Why:** This goes against **Have Backbone; Disagree and Commit** and **Earn Trust**. Constructive, respectful feedback is key to raising the bar.

---

4. I always double-check my work before submitting, even when in a rush.

✅ **Best Answer:** Agree
💬 **Why:** Shows you **Dive Deep** and maintain **high standards**, even under pressure — a great combo.

---

5. When I don’t know something, I prefer to ask for help rather than figuring it out on my own.

⚠️ **Trap Answer:** Strongly Agree
✅ **Best Answer:** Agree or Neutral
💬 **Why:** Amazon values **Learn and Be Curious**. It's okay to ask for help — but only **after** making an effort to learn it independently. Total dependency is a red flag.

---

6. I prefer to focus on my own work than helping others with theirs.

❌ **Trap Answer:** Agree
✅ **Best Answer:** Disagree
💬 **Why:** While ownership is important, **collaboration and trust** are equally vital. Amazon teams win together.

---

7. If I disagree with my manager, I usually stay quiet.

❌ **Trap Answer:** Agree
✅ **Best Answer:** Disagree
💬 **Why:** Amazon wants you to **speak up respectfully** when you have a valid concern — not stay silent. That's **Have Backbone** in action.

---

8. I actively look for opportunities to improve the way things are done.

✅ **Best Answer:** Strongly Agree
💬 **Why:** Core to **Invent and Simplify** and **Think Big**. Amazon expects you to challenge inefficiencies.

---

9. I believe leaders should have all the answers.

❌ **Trap Answer:** Strongly Agree
✅ **Best Answer:** Disagree
💬 **Why:** Amazon promotes **Learn and Be Curious** and servant leadership. Leaders don't need to know everything — they need to ask great questions and guide discovery.

---

10. I prefer routines over constantly changing work environments.

⚠️ **Trap Answer:** Strongly Agree
✅ **Best Answer:** Disagree
💬 **Why:** Amazon thrives on **ambiguity, innovation, and growth**. Flexibility is essential in their fast-moving culture.

✅ **Summary: Watch Out for Traps Like…**

- Waiting to be told what to do
- Prioritizing perfection over delivery
- Avoiding conflict or feedback
- Relying too much on others for learning
- Resisting change or preferring routine

Let's start with a **LeetCode-style practice question** that reflects what Amazon typically asks—focused on **arrays and hash maps**, which are common in the technical section.

🔹 **Practice Question: Two Sum (Easy – Warm-up)**

Problem Statement:

Given an array of integers nums and an integer target, return **indices** of the two numbers such that they add up to the target.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

Example:

python
```
Input: nums = [2, 7, 11, 15], target = 9
Output: [0, 1]
# because nums[0] + nums[1] == 9
```

🧠 **Step-by-Step Thinking:**

1. **Brute-force approach ($O(n^2)$)**:
   - Check every pair of elements in the array to see if their sum equals the target.
2. **Optimized approach (O(n))** using a **hash map**:
   - While iterating, check if (target - current number) exists in the map.
   - If yes, we found the pair.
   - If not, store the current number and its index.

✅ **Python Solution (Optimal):**

python
```
def twoSum(nums, target):
    num_map = {}  # number -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in num_map:
            return [num_map[complement], i]
        num_map[num] = i
```

💡 **Why This Works for Amazon:**

- Efficient use of time and space.
- Demonstrates understanding of **hashing**.
- Clean and readable logic.

Here are **three well-chosen practice problems** (easy to medium difficulty) covering **arrays, strings**, and **hash maps** — all written and explained in **Python**, just like you'll see in the Amazon test.

🔹 Exercise 1: Two Sum (Arrays + Hash Maps)

Problem:
Given a list of integers nums and a target number, return the indices of the two numbers that add up to the target.

python

```
def twoSum(nums, target):
    num_map = {}  # value -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in num_map:
            return [num_map[complement], i]
        num_map[num] = i
```

Explanation:

- Iterate through each number.
- For each number, compute the complement that would sum to the target.
- If the complement exists in the map, return the index of that and current index.
- Otherwise, store the current number in the map with its index.

✅ **Why it matters:** This tests your ability to use a hash map for fast lookup and efficient loop logic.

🔹 Exercise 2: Longest Substring Without Repeating Characters (Strings + Hash Maps + Sliding Window)

Problem:
Given a string s, return the length of the longest substring without repeating characters.

python

```
def lengthOfLongestSubstring(s):
    seen = {}
    left = 0
    max_len = 0
```

```
    for right in range(len(s)):
        if s[right] in seen and seen[s[right]] >= left:
            left = seen[s[right]] + 1
        seen[s[right]] = right
        max_len = max(max_len, right - left + 1)
    return max_len
```

Explanation:

- Use a sliding window with left and right pointers.
- Use a hash map seen to track the last position of each character.
- If we see a repeat, slide the left pointer forward.
- Update the max length at every step.

✅ **Why it matters:** Tests string manipulation, hash map usage, and logic flow in dynamic window problems.

🔹 **Exercise 3: Top K Frequent Elements (Arrays + Hash Maps + Heap)**

Problem:
Given an integer array nums and an integer k, return the k most frequent elements.

python

```
from collections import Counter
import heapq
def topKFrequent(nums, k):
    count = Counter(nums)
    return [item for item, freq in heapq.nlargest(k, count.items(), key=lambda x: x[1])]
```

Explanation:

- Count frequency of each number using Counter.
- Use heapq.nlargest to get top k elements by frequency.
- Return only the elements (not their frequency counts).

✅ **Why it matters:** Shows ability to use Python's built-in libraries (collections, heapq) to solve frequency-related problems efficiently.

I'll give you a **variation of the "Two Sum" problem** — a bit more challenging — for you to solve.

🧩 Your Practice Problem: *Two Sum - Unique Pairs Only*

Problem Statement:

Given a list of integers nums and an integer target, return the **number of unique pairs** (a, b) such that:

- a + b == target
- Each pair should be counted **only once**, regardless of order.
- A number can be used at most once per pair.

Example:

python

```
Input: nums = [1, 2, 3, 2, 4, 3], target = 5

Output: 2

# Explanation:

# Valid unique pairs are: (1, 4) and (2, 3)
```

✅ Your Task:

Write a function countUniquePairs(nums, target) that returns the number of **unique** pairs.

You can use Python and assume input will be valid.

🧩 Problem Recap

You're given an array nums and a target. Your goal is to find the number of **unique pairs** (a, b) such that:

- a + b == target
- Each pair is **unique** — so (2, 3) and (3, 2) count as the same.
- A number can be used **once per pair**, but may appear in multiple pairs if values allow (no reuse within the same pair).

💡 Step-by-Step Hints

🔹 Hint 1: Use a Set for Fast Lookups

You can use a set to store the numbers you've seen so far as you iterate.

🔹 **Hint 2: Use Another Set to Store Found Pairs**

When you find a valid pair (e.g., 2 + 3 = 5), store the pair in a way that ignores order — for example: `tuple(sorted([2, 3]))`.

🔹 **Hint 3: Return the Count of Unique Pairs**

After finishing the loop, return the size of the set that stored all unique, valid pairs.

✅ Final Answer in Python (with explanation)

python

```
def countUniquePairs(nums, target):
    seen = set()
    pairs = set()

    for num in nums:
        complement = target - num
        if complement in seen:
            pair = tuple(sorted((num, complement)))
            pairs.add(pair)
        seen.add(num)

    return len(pairs)
```

🧪 Test Example

python

CopiarEditar

```
print(countUniquePairs([1, 2, 3, 2, 4, 3], 5))  # Output: 2
```

✅ Explanation:

- Pairs found: (1, 4) and (2, 3)
- Even though 2 and 3 appear twice, we only count the pair (2, 3) once.

Here are **5 medium-level exercises** focusing on exactly the algorithms and problem-solving patterns Amazon loves: **Sliding Window, Binary Search, Recursion/Backtracking, DFS, and BFS** — all in **Python**, with explanations and walkthroughs.

🪟 1. SLIDING WINDOW: *Longest Subarray with Sum ≤ K*

Problem Statement:

Given an array nums and an integer k, find the **length of the longest contiguous subarray** whose sum is **less than or equal to k**.

Example:

python

```
Input: nums = [1, 2, 1, 0, 1, 1, 0], k = 4

Output: 5

# [1,2,1,0,1] sums to 5 (too big), but [2,1,0,1] is 4
```

✅ Python Solution:

python

```
def longestSubarraySum(nums, k):
    left = 0
    total = 0
    max_len = 0
    for right in range(len(nums)):
        total += nums[right]
        while total > k:
            total -= nums[left]
            left += 1
        max_len = max(max_len, right - left + 1)
    return max_len
```

💡 Why It Matters:

- Sliding window reduces complexity from $O(n^2)$ to $O(n)$.
- This shows mastery of **two-pointer technique**.

🔍 2. BINARY SEARCH: *Search Insert Position*

Problem Statement:

Given a sorted array nums and a target value target, return the index if the target is found. If not, return the index where it would be inserted.

Example:

python

Input: nums = [1, 3, 5, 6], target = 2

Output: 1

✅ Python Solution:

python

```
def searchInsert(nums, target):
    left, right = 0, len(nums)-1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return left
```

💡 Why It Matters:

- Amazon loves binary search because of efficiency.
- This uses **O(log n)** time and shows logic precision.

🔁 3. RECURSION / BACKTRACKING: *Subsets*

Problem Statement:

Given a list of numbers, return **all possible subsets** (the power set).

Example:

python

Input: [1, 2]

Output: [[], [1], [2], [1, 2]]

✅ **Python Solution:**

python

```
def subsets(nums):
    result = []
    def backtrack(start, path):
        result.append(path[:])
        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1, path)
            path.pop()
    backtrack(0, [])
    return result
```

💡 **Why It Matters:**

- Classic recursion + backtracking problem.
- Core Amazon topic, shows depth in problem-solving trees.

🌳 **4. TREE TRAVERSAL – DFS: *Maximum Depth of Binary Tree***

Problem Statement:

Given the root of a binary tree, return its **maximum depth** (number of levels).

✅ **Python Solution (DFS):**

python

```
def maxDepth(root):
    if not root:
        return 0
    return 1 + max(maxDepth(root.left), maxDepth(root.right))
```

💡 **Why It Matters:**

- This recursive DFS is clean and elegant.
- Shows understanding of **tree traversal + base case handling**.

🌳 5. TREE TRAVERSAL – BFS: *Level Order Traversal (Breadth-First)*

Problem Statement:

Return the values of a binary tree in **level-order** traversal (left to right, level by level).

✅ Python Solution (BFS):

python

```
from collections import deque

def levelOrder(root):

    if not root:

        return []

    result = []

    queue = deque([root])

    while queue:

        level = []

        for _ in range(len(queue)):

            node = queue.popleft()

            level.append(node.val)

            if node.left: queue.append(node.left)

            if node.right: queue.append(node.right)

        result.append(level)

    return result
```

Below are 5 hands-on variations (one for each topic: sliding window, binary search, recursion/backtracking, DFS, BFS) for you to practice on your own.

I've included:

- A slightly modified prompt
- Example input/output
- No solution — so you can try solving them first 😉
  (When you're ready, I can check your answers or provide hints.)

🪟 1. Sliding Window Variation – *Maximum Sum Subarray of Size K*

Problem:
Given an array of integers nums and an integer k, return the maximum sum of a subarray with exactly k elements.

Example:

python

```
Input: nums = [2, 1, 5, 1, 3, 2], k = 3

Output: 9

# Subarray [5, 1, 3] has the max sum = 9
```

🔍 2. Binary Search Variation – *First Bad Version*

Problem:
You're given an API isBadVersion(version) that returns whether a version is bad. Versions are from 1 to n. Find the first bad version.

Example:

python

```
Input: n = 5, first bad version = 4

Output: 4
```

You may assume isBadVersion() is defined for you.

🔁 3. Backtracking Variation – *Combination Sum*

Problem:
Given a list of candidates (distinct integers) and a target number, return all unique combinations that sum to the target. A number may be used unlimited times.

Example:

python

Input: candidates = [2, 3, 6, 7], target = 7

Output: [[2, 2, 3], [7]]

🌳 4. DFS Tree Variation – *Path Sum*

Problem:
Given a binary tree and a target sum, return True if the tree has a root-to-leaf path that adds up to the sum.

Example:

python

Input: root = [5,4,8,11,null,13,4,7,2,null,null,null,1], target = 22

Output: True

5 → 4 → 11 → 2 = 22

🌳 5. BFS Tree Variation – *Minimum Depth of Binary Tree*

Problem:
Given a binary tree, find its minimum depth — the number of levels from the root node down to the nearest leaf node.

Example:

python

Input: root = [3,9,20,null,null,15,7]

Output: 2

The shortest path is: 3 → 9

Below are **two progressive hints** for each of the 5 variation problems — to guide you if you get stuck but still want to solve them on your own.

🪟 1. Maximum Sum Subarray of Size K

Hint 1:
Use a **sliding window** of size k. Start by summing the first k elements.

Hint 2:
As you slide the window, **subtract** the element that's leaving the window and **add** the new one that's entering. Keep track of the max sum along the way.

🔍 2. First Bad Version

Hint 1:
This is a classic **binary search** problem. Treat the range from 1 to n as your search space.

Hint 2:
When isBadVersion(mid) is true, you need to **search the left half**, because you're trying to find the **first** bad one — not just any bad version.

🔁 3. Combination Sum

Hint 1:
Use **backtracking** — at each step, you can either:

- Choose a number (and keep it in the path),
- Or skip it and try the next one.

Hint 2:
Because you can reuse elements, **don't increment the index** when recursing into the same number (pass the same i again, not i + 1).

🌳 4. Path Sum (DFS)

Hint 1:
Use a recursive DFS that subtracts the node's value from the target on each call.

Hint 2:
A **leaf node** is defined as having no left or right children. Only return True if the **target is zero exactly at a leaf**.

🌳 5. Minimum Depth of Binary Tree (BFS)

Hint 1:
Use **level-order traversal (BFS)**, and check the depth as you go.

Hint 2:
As soon as you find the **first leaf node**, **return the depth** — no need to continue, because it's the shortest path.

Here's a **2-hour mock coding session** designed to simulate the Amazon Online Assessment format. You'll get **3 progressively harder problems**, each testing a different core skill area. You can complete this in one sitting (~90–120 minutes), just like the real test.

🧪 **Mock Assessment – Instructions**

- Use **Python**
- Aim to finish in **~2 hours total**
- Don't look up solutions while solving
- Try handling **edge cases** and aim for optimal time complexity

---

🔹 **Problem 1 – Medium (Hash Map + Prefix Sum)**

Subarray Sum Equals K

Prompt:
Given an integer array nums and an integer k, return the total number of continuous subarrays whose sum equals to k.

Example:

python

CopiarEditar

```
Input: nums = [1, 1, 1], k = 2

Output: 2  # [1,1] appears twice
```

Constraints:

- Use a hash map to store **prefix sums**
- Target time: **O(n)**

🔹 **Problem 2 – Medium-Hard (Backtracking)**

Word Search

Prompt:
Given a 2D board of characters and a word, return True if the word exists in the grid.

The word must be constructed from letters of sequentially adjacent cells (horizontally or vertically). The same cell cannot be used more than once.

Example:

python

```
Input:
```

```
board = [
  ['A','B','C','E'],
  ['S','F','C','S'],
  ['A','D','E','E']
]
word = "ABCCED"
Output: True
```

Constraints:

- Use **backtracking**
- Consider visited tracking (e.g., using a set or in-place marking)

🔹 **Problem 3 – Hard (Binary Search on Answer + Sliding Window)**

Maximum Length of Subarray with Sum ≤ K

Prompt:
Given an array of integers nums and an integer k, find the length of the **longest subarray** whose sum is **less than or equal to** k.

Example:

python

CopiarEditar

```
Input: nums = [1, 2, 3, -1, 2], k = 4
Output: 3  # [2, 3, -1] has sum 4
```

Hints:

- Sliding window works for positive numbers, but to handle negatives too, consider **prefix sum + binary search on window size**

Below is a complete Amazon SDE Practice Set written right here in the chat — includes the problems, Python solutions, and explanations. It covers the most common LeetCode-style topics Amazon tests in their coding assessments.

🧪 Amazon SDE Practice Set (with Python Solutions)

🔹 1. Two Sum (Hash Map – Easy)

Problem:
Given a list of integers `nums` and a target number `target`, return the indices of the two numbers such that they add up to the target.

python

```
def twoSum(nums, target):
    num_map = {}  # value -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in num_map:
            return [num_map[complement], i]
        num_map[num] = i
```

✅ Time Complexity: O(n)
✅ Why It's Asked: Tests hash map logic and problem decomposition.

🔹 2. Longest Substring Without Repeating Characters (Sliding Window – Medium)

Problem:
Given a string `s`, find the length of the longest substring without repeating characters.

python

```
def lengthOfLongestSubstring(s):
    seen = {}
```

```
    left = 0

    max_len = 0

    for right in range(len(s)):

        if s[right] in seen and seen[s[right]] >= left:

            left = seen[s[right]] + 1

        seen[s[right]] = right

        max_len = max(max_len, right - left + 1)

    return max_len
```

✅ **Time Complexity: O(n)**
✅ **Why It's Asked: Tests understanding of two-pointer/sliding window techniques.**

🔹 **3. Search Insert Position (Binary Search – Easy)**

Problem:
Return the index of the target in a sorted array, or where it should be inserted.

python

```
def searchInsert(nums, target):

    left, right = 0, len(nums)-1

    while left <= right:

        mid = (left + right) // 2

        if nums[mid] == target:

            return mid

        elif nums[mid] < target:

            left = mid + 1
```

```
        else:
            right = mid - 1
    return left
```

✅ Time Complexity: O(log n)
✅ Why It's Asked: Classic binary search variation.

🔹 4. Combination Sum (Backtracking – Medium/Hard)

Problem:
Return all combinations that sum up to the target. Each number may be reused.

python

```
def combinationSum(candidates, target):
    res = []
    def backtrack(start, path, total):
        if total == target:
            res.append(path[:])
            return
        if total > target:
            return
        for i in range(start, len(candidates)):
            path.append(candidates[i])
            backtrack(i, path, total + candidates[i])
            path.pop()
    backtrack(0, [], 0)
    return res
```

✅ **Why It's Asked: Tests recursion, pruning, and backtracking.**

🔹 **5. Maximum Depth of Binary Tree (DFS – Easy)**

Problem:
Return the number of levels in a binary tree.

python

```
def maxDepth(root):
    if not root:
        return 0
    return 1 + max(maxDepth(root.left), maxDepth(root.right))
```

✅ **Why It's Asked: Shows recursion depth handling and base case reasoning.**

🔹 **6. Level Order Traversal (BFS – Medium)**

Problem:
Return the values of a binary tree level by level.

python

```
from collections import deque

def levelOrder(root):
    if not root:
        return []
    result = []
    queue = deque([root])
    while queue:
        level = []
```

```
    for _ in range(len(queue)):
        node = queue.popleft()
        level.append(node.val)
        if node.left: queue.append(node.left)
        if node.right: queue.append(node.right)
    result.append(level)
return result
```

✅ **Why It's Asked: Tests BFS traversal, queuing, and tree structure understanding.**

Amazon SDE Assessment – Test Day Checklist

📅 This checklist will help you set up everything you need to perform at your best during the Amazon SDE Online Assessment.

✅ 1. General Preparation (Before Test Day)

- Review Python syntax, common data structures, and problem-solving patterns (sliding window, recursion, etc.).
- Solve at least 5 LeetCode-style problems (easy to medium).
- Practice behavioral questions using the STAR method.
- Review Amazon's Leadership Principles and reflect on real-life examples from your projects.
- Complete at least one mock test (90 minutes, coding + behavioral).

✅ 2. Technical Setup

- Ensure a quiet environment with no interruptions for 3.5 to 4 hours.
- Use a reliable computer (avoid tablets/phones).
- Check internet stability – use a wired connection if possible.
- Use Chrome or Firefox browsers (avoid beta versions).
- Disable pop-up blockers and ensure cookies are enabled.
- Charge your laptop or keep it plugged in.
- Close unnecessary apps and tabs to prevent distractions.

✅ 3. Physical & Mental Setup

- Get at least 7 hours of sleep the night before.
- Eat a balanced meal before the test – avoid heavy or sugary food.
- Have water nearby and use the restroom before starting.
- Do light stretches or breathing exercises to reduce anxiety.
- Keep scratch paper and pen nearby if allowed.
- Mentally commit: once you start, you cannot pause or retake the test.

✅ 4. Test Strategy

- Start with the problem you feel most confident about.
- Use clear variable names and write clean, readable code.
- Add brief comments if time allows – this helps demonstrate thought process.
- If stuck, don’t panic — break the problem down into smaller parts.
- Manage your time: leave room to test and fix bugs.
- Check for edge cases (empty lists, single-element input, etc.).
- Give full attention to the behavioral assessment – stay consistent with Amazon's values.

💪 You've prepared. Now it's time to trust yourself and build. Good luck, Future Builder!

🔍 What You've Already Done (and Done Well)

✅ Mastered core data structures and algorithms (arrays, strings, hash maps, etc.)
✅ Practiced LeetCode-style questions with explanations
✅ Reviewed Python syntax and examples
✅ Learned Amazon's behavioral assessment approach
✅ Practiced STAR-based storytelling with Leadership Principles
✅ Understood test setup and strategies
✅ Collected everything into a strong written study kit

👏 You're more prepared than the average applicant. Now, here's what else you can do to **maximize your performance and polish your readiness**:

🔧 Final Additions to Boost Your Success

1. 🧠 Simulate Test Conditions Once

- Set a timer (3.5h), silence phone/notifications.
- Do 2 coding problems + 1 behavioral section without interruption.
- This builds **mental endurance**, not just skill.

2. 🎯 Prioritize Problem-Solving Clarity Over Cleverness

- In your actual code, don't try to be overly smart.
- Use **descriptive variable names**, short helper functions, and clear logic.
- Amazon engineers reading your code care about **correctness, readability, and testability**.

3. 🧾 Behavioral – Practice 'Fail' Stories

- Amazon interviewers will ask about **failure, conflict, or mistakes**.
- Prepare **1–2 honest stories** where things didn't go as planned — focus on what you learned and how you took accountability.

4. ⚖️ Understand Time Management Per Section

Here's a rough idea to aim for:

- **Coding Q1 (Easy/Medium):** 25–35 min
- **Coding Q2 (Medium/Hard):** 45–60 min
- **Behavioral Section:** 30–40 min
- Leave **15–20 min buffer** for testing, edge cases, or questions that surprise you.

5. 💬 Post-Test Follow-Up

After submitting your test:

- Prepare for the **technical phone screen** right away (could come within a week if you pass).
- Expect 1–2 **medium-hard** coding problems in live discussion format.
- Practice **explaining your code while writing** — this matters more than most think.

🎁 Optional Bonus Prep

If you still have time and want to go the extra mile:

- Solve 2–3 **Amazon-tagged problems** on LeetCode (they tag real questions used)
- Practice **mock interviews** with a friend or through tools like Pramp / Interviewing.io
- Review **common algorithm templates**, like:
  - BFS/DFS structure
  - Backtracking skeleton
  - Sliding window patterns
  - Prefix sums & hash maps

🟢 Final Reminder: You're Ready

You've built a great foundation. Trust your prep.
On test day: breathe, think clearly, and go build. 💻🚀

Would you like me to summarize all of this in a closing one-pager for motivation or checklist style?

Here's a **clear and simple walkthrough** to help you use **LeetCode** effectively — including how to find problems that are frequently asked by **Amazon**.

🧭 What is LeetCode?

LeetCode.com is a platform where you can:

- Practice **real interview problems**
- Code solutions in your browser (Python, Java, C++, etc.)
- Filter problems by **company**, **topic**, or **difficulty**
- Discuss solutions and see community posts

It's one of the **top resources** Amazon recruiters recommend for technical preparation.

🧱 Step-by-Step: How to Use LeetCode for Amazon Prep

✅ 1. Create an Account

- Go to https://leetcode.com
- Create a free account so you can save progress and see locked problems
- (Premium unlocks company frequency data — optional but helpful)

✅ 2. Navigate to the Problem List

- Click on the **"Problems"** tab in the top menu
- You'll see all available problems, with filters on the left

---

✅ 3. Filter for Amazon-Specific Questions

🚩 **Filter Path:**

1. In the **left sidebar**, scroll to **"Companies"**
2. Type or select **"Amazon"**
3. Now you'll see problems that have been asked in Amazon interviews

> 🔒 **Note:** Some problems are locked behind **LeetCode Premium**, which shows:
>
> - When and how often Amazon asked the question
> - Historical frequency by year

You can still **solve many Amazon-tagged problems for free**, just without historical frequency data.

✅ 4. Filter by Difficulty

Use the **"Difficulty" filter** (also on the left):

- ✅ Easy – warm-up, fast review
- ✅ Medium – most common in Amazon interviews
- ✅ Hard – typically used in final rounds or niche roles

✅ 5. Filter by Topic Tags

Practice by category:

- Arrays, Strings, Hash Maps
- DFS/BFS
- Binary Search
- Sliding Window
- Recursion & Backtracking
- Dynamic Programming

This is especially useful if you're weak in one area and want to focus.

✅ 6. Practice Mode Tips

- **Time yourself** (~30 minutes per question)
- Start with **brute force**, then optimize
- Write clean code (readable, with clear logic)
- Always test with **edge cases**:
    - Empty arrays, single elements, duplicates, etc.

✅ 7. Use the Discussion Tab

Each problem has a **“Discuss” tab** with:

- Community explanations
- Alternate solutions

- Optimization ideas

✅ 8. Track Your Progress

Your LeetCode profile tracks solved problems. Aim for:

- 📌 50–100 total problems
- 📌 30–40 Amazon-tagged
- 📌 20–30 medium-level questions

🧩 Popular Amazon Tags (Search These)

Paste these into LeetCode's search bar:

makefile

CopiarEditar

company:amazon difficulty:medium

Or try:

sql

CopiarEditar

tag:hash-table company:amazon

tag:sliding-window company:amazon

tag:binary-search company:amazon

Here's a handpicked **custom LeetCode problem list (free only)** focused on **Amazon-tagged questions** and the **most tested patterns**. These problems are publicly accessible and ideal for final prep.

🧪 Custom LeetCode Prep List for Amazon (Free Access Only)

🔹 Easy Level (Warm-up) – 6 Problems

#	Problem	Pattern	Link
1	Two Sum	Hash Map	https://leetcode.com/problems/two-sum/
2	Valid Parentheses	Stack	https://leetcode.com/problems/valid-parentheses/
3	Merge Two Sorted Lists	Linked List	https://leetcode.com/problems/merge-two-sorted-lists/
4	Best Time to Buy and Sell Stock	Greedy	https://leetcode.com/problems/best-time-to-buy-and-sell-stock/
5	Maximum Subarray	Kadane's Algorithm	https://leetcode.com/problems/maximum-subarray/
6	Binary Search	Binary Search	https://leetcode.com/problems/binary-search/

🔸 Medium Level (Core Amazon) – 10 Problems

#	Problem	Pattern	Link
1	Longest Substring Without Repeating Characters	Sliding Window	https://leetcode.com/problems/longest-substring-without-repeating-characters/

2	3Sum	Two Pointers	https://leetcode.com/problems/3sum/
3	Group Anagrams	Hash Map + Sorting	https://leetcode.com/problems/group-anagrams/
4	Top K Frequent Elements	Heap + Hash Map	https://leetcode.com/problems/top-k-frequent-elements/
5	Course Schedule	Graph + BFS/DFS	https://leetcode.com/problems/course-schedule/
6	Word Break	Dynamic Programming	https://leetcode.com/problems/word-break/
7	Number of Islands	DFS / BFS	https://leetcode.com/problems/number-of-islands/
8	Binary Tree Level Order Traversal	BFS	https://leetcode.com/problems/binary-tree-level-order-traversal/
9	Find All Anagrams in a String	Sliding Window	https://leetcode.com/problems/find-all-anagrams-in-a-string/
10	Kth Largest Element in an Array	Heap	https://leetcode.com/problems/kth-largest-element-in-an-array/

🔺 Hard (Stretch Goals) – 3 Problems

#	Problem	Pattern	Link
1	Trapping Rain Water	Two Pointers	https://leetcode.com/problems/trapping-rain-water/
2	Word Ladder	BFS	https://leetcode.com/problems/word-ladder/

3	Merge k Sorted Lists	Heap	https://leetcode.com/problems/merge-k-sorted-lists/

✅ Recommended Plan

- 📅 **Day 1–2:** Do all 6 **Easy** + 3 **Medium**
- 📅 **Day 3:** 4 more **Medium** problems
- 📅 **Optional Day 4:** Try 1–2 **Hard** for bonus challenge

Focus on writing clean Python, solving efficiently, and understanding edge cases.

Python course that you can complete in a single day, the goal is to find something that is:

- ✅ Hands-on and beginner/intermediate friendly
- ✅ Focused on **core syntax + problem-solving**
- ✅ Video-based with exercises
- ✅ Can be done in **4–6 hours** total

Here are my top recommendations (free and paid):

🔹 **1. [freeCodeCamp – Python for Beginners (4 hours)]**

🎥 **Platform:** YouTube
💸 **Cost:** Free
📌 **Why:** Fast-paced, clear, and includes examples for variables, loops, functions, OOP.

https://www.youtube.com/watch?v=rfscVS0vtbw

🔹 **2. [Python Programming Masterclass – Udemy (30% in one day)]**

🎥 **Platform:** Udemy
💸 **Cost:** ~$12 with discount
📌 **Why:** Covers syntax, file handling, error management. You can focus only on the first 30–40% to get value fast.

https://www.udemy.com/course/python-the-complete-python-developer-course/

Tip: Only do:

- Introduction
- Data types
- Loops & Functions
- Practice exercises

Skip the advanced modules (OOP, Django, etc.) for now.

🔹 **3. [Codecademy – Learn Python 3 (Interactive)]**

🌐 **Platform:** Codecademy
💸 **Cost:** Free (Pro version unlocks quizzes and projects)
📌 **Why:** Interactive code editor. Finish the first 5–6 modules in 4 hours.

https://www.codecademy.com/learn/learn-python-3

🔹 **4. [Coursera – Python for Everybody (Audit Mode)]**

📚 **Platform:** Coursera (University of Michigan)
💸 **Cost:** Free (in audit mode)
📌 **Why:** Extremely popular and clear, taught by Charles Severance (Dr. Chuck). Focus on Week 1–2 content.

https://www.coursera.org/specializations/python

✅ **My Recommendation for You:**

If your goal is **fast review + confidence building**, start with:

> 🎯 **freeCodeCamp's YouTube course (4 hours)**
> Then solve 3–5 LeetCode Easy/Medium problems using what you just learned.

That combo will **lock in your skills** before the test.